What Does the SDN Controller Actually See? Controller-Observable Telemetry in InSDN's Own OpenFlow Control Channel

Adhyan Jain [Affiliation]

Abstract*—SDN intrusion-detection research is evaluated mostly on benchmark CSVs of CICFlowMeter 5-tuple flow records, computed offline from packet captures at the attacked host. Whether an SDN controller can observe anything resembling these records is rarely checked. We check it inside the benchmark itself. InSDN, a widely used SDN intrusion dataset, also published packet captures of its ONOS controller's OpenFlow 1.3 control channel; we parse all 44 of them (11 benign and 33 attack sessions across seven attack classes). In every flow-statistics reply, and in all 3 827 FlowMods, in these captures the controller's flow entries match only on `eth_type` or `(in_port, eth_src, eth_dst)`; none matches IP addresses or transport ports, so the dataset's labelled 5-tuple flows are not counters the controller can read. What the controller does receive is a dense PacketIn stream: in 9 of 12 attack scenarios, between 0.68 and 1.29 PacketIns per data packet seen at the target. Using only this controller-observable view, small models reach a balanced accuracy of 0.946 on unseen captures under capture-grouped cross-validation (against 0.616 for a PacketIn-rate threshold) and 0.89–1.00 on all six withheld attack families (best single-feature threshold 0.70–0.97); logistic regression matches the neural network overall. We then show that these numbers cannot be read as attack-detection accuracy: InSDN records benign and attack traffic in separate sessions and its labels cover only part of the controller-visible traffic, so within attack sessions a trivial "any IP PacketIn" rule scores 0.995 under strict labels, and our session probes cannot separate session identity from unlisted attack traffic. We report this as a validity threat for controller-side evaluation on InSDN. Finally, a benign-traffic Mininet/Ryu case study (10 runs per mode on one switch, 10 on a three-switch chain) shows that the forwarding application sets what the controller observes: per-flow forwarding produced about 10 400 TCP PacketIns per run against about 12 for MAC-pair forwarding (at least 665× in every paired run, for both timeouts; at least 391× on three switches), and a 2 s idle timeout reduced the IP-specific entries a 1 s poller sees from 153.1 ± 2.4 to 36.9 ± 0.5 per reply. A first campaign with a flawed application, in which ping and ARP entries shadowed TCP, understated the PacketIn effect by more than two orders of magnitude; we report both. Ryu's unmodified `simple_switch_13` application, run as a second controller in five captures, also installed only MAC-pair entries (86 of 91 FlowMods; none with IP or transport fields).

***Index Terms*—Software-defined networking, intrusion detection, OpenFlow, ONOS, InSDN, telemetry, evaluation methodology**

I. Introduction

Machine-learning intrusion detection for software-defined networks (SDN) is commonly motivated by the controller's global view [1]. In practice, evaluation rarely involves a controller: models are trained and tested on flow records computed by CICFlowMeter [2] from packet captures at the attacked hosts, then described as SDN detectors. InSDN [3] is a prominent example; its CSV release of ∼344k labelled bidirectional 5-tuple flows with 80+ statistical features is widely used as an SDN benchmark.

A controller cannot compute those features for flows it does not see in full. An OpenFlow 1.3 controller [4] observes PacketIn messages for packets that miss the flow table, flow-statistics replies with packet and byte counters for whatever entries its forwarding application installed, and its own FlowMod/PacketOut decisions. Inter-arrival-time moments, per-direction length statistics and flag counts are not in that vocabulary. Earlier work criticised benchmark flow features for diverging from real flow metering [5], [6], [7], argued for standard feature sets [8], and measured the cost of assuming complete flows [9]; a recent dataset records control-plane telemetry on a new testbed [10]. Controller-side detectors based on PacketIn behaviour exist [11], [12], [13], [14], [15], but they are evaluated on their own traffic. We found no work that measures, inside InSDN, what the controller observed, or that evaluates detection restricted to that view (our searches were conducted in October 2026; they are evidence, not proof, of absence).

InSDN's authors also captured the ONOS [16] controller interface during every scenario. We analyse these captures and make five contributions:

1) **A measured observability gap (§IV).** Across all flow-statistics replies, only two match structures occur and none involves IP or transport fields (also true of all 3 827 FlowMods); in 9 of 12 scenarios the controller receives 0.68 to 1.29 PacketIns per target packet.
2) **A reproducible control-channel pipeline (§V).** An OpenFlow 1.3 parser and window featuriser for the 44 captures with reported parser loss, and window labels grounded in InSDN's own flow labels.
3) **Detection from the controller's view, as a feasibility result (§VI).** Evaluated on unseen captures, withheld attack families and across the two target deployments, against threshold heuristics and simpler models, over 10 seeds.
4) **A validity analysis (§VII).** Under InSDN's design, the detection scores cannot be separated from session recognition and label coverage; we quantify what a trivial rule achieves and what our probes can and cannot show.
5) **A forwarding-application case study (§VIII).** In a Mininet/Ryu testbed with benign traffic, per-flow for-

warding produces about 10 400 TCP PacketIns per run against about 12 for MAC-pair forwarding, and the idle timeout sets how many entries a poller sees; a first campaign with a flawed application understated this effect.

II. Background and Related Work

What an OpenFlow controller observes. Under reactive forwarding, the first packet of an unmatched flow reaches the controller as a PacketIn carrying the packet's headers; the controller answers with a PacketOut and optionally installs an entry via FlowMod. It can poll per-entry packet and byte counters with multipart flow-statistics requests [4]. Which entries exist, and hence what the counters mean, is decided by the forwarding application, not by the IDS.

Benchmark realism and evaluation pitfalls. Sommer and Paxson [17] and Arp et al. [18] catalogue pitfalls of ML-based detection, including spurious correlations. Engelen et al. [5] found labelling and flow-construction errors in CICIDS2017; Pekar and Jozsa studied dataset integrity [7] and train/test flow completeness [9]; Pinto et al. [6] showed detectors depend on the flow exporter; Sarhan et al. [8] argued for standard NetFlow feature sets. These concern flow-record features; none asks what an SDN controller can observe.

Controller-side detection and control-plane data. PacketIn and flow-table signals have long been used to detect flooding at the controller [11], [12] and to protect it [13], [14], [15]. Arévalo-Herrera et al. [10] release a dataset that pairs traffic features with control-plane metrics (PacketIn and FlowMod counts) collected on an OpenDaylight testbed. Their ablation reports that removing the OpenFlow features lowers link-fabrication F1 from 0.97 to 0.19 on that dataset, which supports the premise that control-plane telemetry carries signal for SDN-specific attacks; our results neither confirm nor contradict it, because we study volumetric and scan traffic on a different testbed. The UKMDDoSDN dataset [19] (Ryu, DDoS) ships PCAPs, but the two we fetched (SYN and UDP flood, 310 k packets in total) contain no OpenFlow packets at all (Appendix A), so they cannot support a control-channel analysis. Our work is complementary: we do not introduce a new dataset, but characterise and use the controller-side captures already shipped with InSDN, so results remain comparable with the large body of InSDN work.

III. Threat Model and Scope

Defender. The detector runs at (or beside) an SDN controller and may read only what the OpenFlow 1.3 control channel carries: PacketIn, PacketOut, FlowMod, FlowRemoved and flow-statistics messages. It has no tap on host links and no packet captures from the data plane. We assume the controller's forwarding application is fixed and known.

Attacker. The attacker is an end host that sends volumetric (DoS/DDoS), scanning, brute-force, exploit, web or botnet traffic through the switch. We do not consider an attacker who attacks the controller or the control channel itself (e.g. spoofed PacketIn floods aimed at saturating it), who knows the detector, or who adapts traffic to mimic benign PacketIn statistics; the evaluation is therefore a measurement of what is observable and separable in recorded benchmark traffic, not a robustness claim.

Out of scope. Inline operation, mitigation, latency, encrypted-payload effects, and attacks on switches. The paper makes claims about (i) what two forwarding applications install, (ii) separability within InSDN's captures, and (iii) validity of that separability as detection evidence.

IV. What the Controller Sees in InSDN

Data. InSDN [3] was generated with Mininet hosts and an Open vSwitch switch controlled by ONOS; attacks target a Metasploitable-2 VM or the OVS host. Besides the CSVs, the authors published PCAP groups (*Normal*, *Metasploitable-2*, *OVS*) recorded at the target and at the controller interface. We downloaded them from the dataset's institutional host; the Metasploitable-2 and OVS archives passed ZIP integrity checks, and for the Normal archive we fetched only the controller captures through HTTP range requests on its ZIP members, each checked against its recorded size. SHA-256 hashes of the 44 captures and the two full archives are released with the code (`capture_hashes.sha256`). The 44 controller captures (`*ONOS*`) are the subject of this paper (Table I).

Granularity. We decoded every OpenFlow 1.3 multipart flow-statistics reply in all 44 captures. Only two match structures occur: `eth_type` (the controller's default punt rules; 80 464 entry observations) and `(in_port, eth_src, eth_dst)` (reactive forwarding; 24 437 observations). No entry matches IP addresses or transport ports. A 5-tuple flow, the unit InSDN labels, is therefore never a counter the controller can read; a MAC-pair entry aggregates every flow between two hosts. The 3 827 FlowMods the controller sent show the same: 3 811 match `(in_port, eth_dst, eth_src)`, 16 match `eth_type`, and none has IP or transport fields, so a short-lived 5-tuple entry cannot have slipped between two polls. This describes the ONOS forwarding configuration used in InSDN; it is not a property of OpenFlow in general.

The PacketIn stream. Table II pairs each attack scenario's target-side captures with its controller captures. In 9 of 12 scenarios the controller receives between 0.68 and 1.29 PacketIns per data packet recorded at the target. Two Metasploitable-2 scenarios show 3.9 and 15.7. The OVS DDoS scenario shows 0.08 (6.5M target packets against 541k PacketIns), consistent with control-channel saturation under a spoofed flood; we did not test this directly. The two sides are separate recordings and their alignment is unverified, so ratios are indicative, not exact per-packet counts. The Normal group has no target-side captures, so we have no such ratio for benign traffic. Because each PacketIn carries the packet's headers, the 5-tuple is visible to the controller per packet, but never as a per-flow counter.

Implication. A model trained on InSDN's CSV consumes statistics (inter-arrival moments, per-direction length moments, flag counts per 5-tuple) that this controller never receives. The deployable signal is the PacketIn header stream plus coarse MAC-pair counters, which we use directly.

V. Controller-Observable Detection

Parsing. We reassemble each TCP direction of the control channel (port 6653), split it into OpenFlow messages by header length, and decode PacketIn (with the embedded Ethernet frame), PacketOut, FlowMod, FlowRemoved and flow-statistics replies. On a header desynchronisation the parser scans forward for the next position that starts a chain of two plausible headers. Over the 44 captures it decoded 7 381 679 messages, resynchronised 134 times and discarded 12 217 bytes; reassembly is in arrival order with sequence-number deduplication only, so these loss figures are a lower bound. (A first version that dropped its whole buffer on every desync discarded 44.6 MB and lost up to $\sim$90% of PacketIns on some DDoS captures; all reported results use the corrected parser.) 182 153 PacketIns are truncated by the switch to a header prefix; their headers still parse and labelling is unaffected.

Features. For each window of $W{=}1$ s we compute 22 controller-observable features: counts of PacketIn, PacketOut, FlowMod, FlowRemoved and statistics replies; from PacketIn frames, the number of distinct source and destination MACs, IPs and destination ports, TCP/UDP/ICMP/ARP fractions, the SYN-only fraction of TCP, mean frame length, and the entropies of source IP and destination port; and, from consecutive statistics replies of the same entry, the number of entries and the sum and maximum of packet and byte deltas. The model input concatenates the current and three previous windows ($\log(1{+}x)$ transformed).

Labels grounded in the dataset (strict). Labelling a window by the capture it came from would mark every quiet window of an attack session as an attack. Instead we label each PacketIn by matching its 5-tuple (either direction) to InSDN's CSV Flow IDs. A window is an attack window iff it contains at least one PacketIn matching an attack flow. In attack captures, a window with no attack match but with PacketIns of unmatched IP traffic is ambiguous and is excluded from training and scoring; benign captures keep all windows. This is restrictive and sparse: the CSV covers only part of each capture, so after exclusion brute force, exploit and web keep only 6, 12 and 12 attack windows, and DDoS 55 of 499 (Table I). As §VII shows, the exclusion also removes exactly the hard negatives of attack sessions.

Detectors and baselines. A two-layer MLP (64–32 units, dropout, class-weighted cross-entropy, 40 epochs) trained on an NVIDIA RTX 4060 GPU; a logistic-regression model on the same inputs; a single-window MLP; ablations to PacketIn-derived or flow-statistics-derived features; and four single-feature thresholds fitted on the training folds (PacketIn rate, a causal per-capture-normalised burst rate, source-IP entropy, destination-port entropy). A gradient-boosted-tree baseline (XGBoost, trained on the GPU) uses the same inputs, and a host-side baseline uses InSDN's own flow-CSV features (below).

Protocol. No capture appears in both training and test. (A) 5-fold cross-validation grouped by capture file and stratified by class; (B) leave-one-attack-family-out, with DoS and DDoS withheld together as "flood"; (C) cross-deployment, training on one target group and testing on the other, with disjoint benign captures; (D) session probes; (E) a label-shuffle sanity check. Every experiment uses 10 seeds (fold assignment and initialisation). We report mean and 95% t-interval over seeds, which reflects fold and initialisation variability on the same 44 captures, not sampling of new testbeds, and a stratified cluster bootstrap over captures for pooled window counts (a CI of pooled window BAcc, dominated by the many benign windows, hence not comparable to the seed mean). Folds are grouped by capture file, not by scenario: variants of one scenario (e.g. SYN, UDP and ICMP DDoS) can fall on both sides of a fold, so (A) is optimistic; (B) and (C) are the scenario-held-out tests. Balanced accuracy (BAcc) is the headline because classes are imbalanced.

VI. Results

Unseen captures (Table III). The controller-observable MLP reaches BAcc 0.946 with a window false-positive rate of 0.001. The PacketIn-rate threshold, the usual SDN flooding heuristic, reaches only 0.616; a likely reason is that benign sessions also produce heavy PacketIn volume, but we cannot verify this against a target-side benign ratio (§IV). Entropy thresholds do better (0.839–0.852) but stay below the learned models. PacketIn features carry the signal (0.943 alone); flow-statistics features alone reach 0.699 at 1 s with a high FPR (0.918 at 5 s), consistent with MAC-pair counters being coarse. A logistic-regression model (0.939) and a single-window MLP (0.934) are close to the full model overall, so history and non-linearity add little here, although logistic regression differs on some withheld families (e.g. botnet 0.803 vs. 0.932). The capture-level bootstrap (Table IV) is wider than the seed intervals and pools windows, so it is dominated by benign windows; there the entropy thresholds look competitive with the learned models, and the seed-level comparisons are the more informative ones. The label-shuffle control (each capture given another capture's window-label sequence) gives BAcc 0.512 ± 0.054 at 1 s and 0.508 ± 0.067 at 5 s, i.e. chance; it shows the pipeline does not read labels from where it should not, and says nothing about label–feature overlap or session identity.

Withheld attack families (Table V). With a family absent from training, the MLP beats the best single-feature threshold on flood (0.936 vs. 0.814), probe (0.997 vs. 0.970) and web (0.904 vs. 0.699), ties on brute force and exploit, and loses on botnet (0.932 vs. 0.965; one capture). The best threshold is chosen per family in hindsight, which favours the baseline, and brute force, exploit and web rest on 6–12 attack windows.

Across deployments (Table VI). Training on OVS-target sessions and testing on Metasploitable-2 sessions gives 0.990 (best threshold 0.947); the reverse gives 0.905 (best threshold 0.866).

Window length. With $W{=}5$ s (strict labels, 10 seeds) the picture is consistent: BAcc 0.958 for the MLP (logistic regression 0.961, single window 0.944) and the PacketIn-rate threshold at 0.527. Flow-statistics features alone improve to 0.918, since a longer window usually spans a counter poll. Cross-deployment gives 0.978 (OVS $\rightarrow$ Metasploitable-2; best threshold 0.910) and 0.895 (reverse; 0.860).

TABLE I
CONTROLLER CAPTURES (1 S WINDOWS). UNDER STRICT LABELS, AMBIGUOUS WINDOWS OF ATTACK CAPTURES ARE EXCLUDED ("KEPT").

Class	Captures	Windows	Kept	Attack win.	PacketIns
Normal	11	33,283	33,283	0	789,569
DoS	11	861	805	456	845,362
DDoS	6	499	184	55	1,340,347
Probe	4	5,968	2,921	1,965	621,986
Brute force	4	604	359	6	5,564
Exploit	4	88	71	12	332
Web	3	636	275	12	1,199
Botnet	1	160	152	83	1,002

TABLE II
TARGET-SIDE TRAFFIC VS. CONTROLLER OBSERVATIONS PER SCENARIO (RATIO = PACKETINS PER TARGET PACKET).

Group	Scenario	Target pkts	5-tuples	PacketIns	Ratio
Metasp.	Brute force	1,311	295	1,574	1.20
Metasp.	DDoS	652,495	651,083	799,253	1.22
Metasp.	DoS	13,078	989	205,198	15.69
Metasp.	Exploit	86	18	332	3.86
Metasp.	Probe	507,650	51,150	510,194	1.01
OVS	Botnet	902	164	1,002	1.11
OVS	DDoS	6,510,923	6,508,402	541,094	0.08
OVS	Probe	86,705	79,021	111,792	1.29
OVS	Brute force	5,892	1,112	3,990	0.68
OVS	App. DoS	299,625	41,738	308,681	1.03
OVS	Net. DoS	304,886	20,979	331,483	1.09
OVS	Web	987	176	1,199	1.21

TABLE III
UNSEEN CAPTURES, GROUPED 5-FOLD CV, W=1 S, STRICT LABELS, 10 SEEDS (MEAN±95% CI OVER SEEDS).

Method	BAcc	Recall	FPR
MLP, all features	0.946±0.016	0.893	0.001
MLP, PacketIn features only	0.943±0.015	0.887	0.002
Logistic regression (history)	0.939±0.016	0.887	0.009
MLP, single window	0.934±0.015	0.870	0.003
Thr. source-IP entropy	0.852±0.029	0.763	0.059
Thr. dst-port entropy	0.839±0.018	0.736	0.059
MLP, flow-stats features only	0.699±0.036	0.683	0.284
Thr. PacketIn burst	0.643±0.008	0.912	0.627
Thr. PacketIn rate	0.616±0.011	0.793	0.561
MLP, within attack sessions	0.942±0.016	0.893	0.008
macro-averaged over captures	0.906±0.009	–	–
Trivial: "any IP PacketIn" (same windows)	0.995	–	–
macro over captures	1.000	–	–

TABLE IV
POOLED WINDOW-COUNT BACC WITH A CAPTURE-LEVEL CLUSTER BOOTSTRAP (2000 RESAMPLES, STRATIFIED BY CLASS).

Method	Pooled BAcc	95% CI
MLP, all features	0.984	[0.926, 0.995]
Logistic regression	0.979	[0.921, 0.991]
MLP, single window	0.979	[0.905, 0.993]
Thr. source-IP entropy	0.923	[0.783, 0.959]
Thr. dst-port entropy	0.926	[0.792, 0.953]
Thr. PacketIn rate	0.467	[0.245, 0.817]

Host-side baseline. For contrast, the usual flow-CSV protocol applied to InSDN's own CICFlowMeter features (343 889 flows, 78 numeric features, benign vs. any attack, XGBoost on the GPU, 10 seeds of stratified 70/30 row splits) reaches BAcc 0.9998 ± 0.00003. This is not a comparison on the same traffic or the same units: rows are flows, not windows; rows of one session fall on both sides of the split; and these features are not available to a controller. It shows that the usual benchmark protocol is far easier than capture-grouped evaluation of the controller view; it should not be read as the controller-side detector being 5 points worse.

VII. VALIDITY: CAN WE TELL ATTACKS FROM SESSIONS?

InSDN's benign and attack traffic were recorded in separate sessions, and its CSV labels cover only part of the traffic visible to the controller. Both can inflate the scores above, a spurious-correlation pitfall [18]. We tested this and report what the evidence does and does not show.

Within-session scoring under strict labels is trivial. Restricting to windows of attack captures that contain at least one PacketIn, the MLP scores BAcc 0.942 pooled and 0.906 macro-averaged over captures (Table III). A trivial rule, "at least one IP PacketIn in the window", scores 0.995 pooled and 1.000 macro on the same windows (2 589 attack and 320 attack-free windows at 1 s; 616 and 22 at 5 s, where it scores 1.000). The reason is the ambiguity mask: it removes attack-session windows with IP traffic not matched by the CSV, leaving attack-free windows that are almost all ARP/LLDP only (317 of 320 have no IP PacketIn). We therefore do not use the strict within-session figure as evidence of detection.

With broader labels the margin over the trivial rule is real but not clean. Under the IP-based label rule (§IX), which keeps all windows, the same trivial rule scores 0.741 pooled and 0.700 macro at 1 s (0.653 and 0.668 at 5 s), while the MLP scores 0.794 and 0.855 (0.715 and 0.782). The MLP is ahead, but this rule labels windows by IP membership, which overlaps with the IP-count features, so the comparison is not independent.

The session probe is too small and too impure to quantify session identity (Table VII). We remove attack-matched PacketIns, drop controller-reply counts, keep windows with at least five remaining PacketIns, and train the MLP to predict whether a window comes from an attack session. It reaches BAcc 0.940 (majority 0.500), but the attack side has 518 windows, of which 424 (82%) contain unmatched-IP PacketIns that may simply be attack traffic the CSV does not list; the stricter variant using only attack-free windows has 6 windows from 2 captures at 1 s (0.950) and 2 windows at 5 s (0.550). Under the IP-based rule the corresponding probe scores 0.806 and 0.783, again with unlisted traffic in the "attack-free" class. We conclude only that the available data cannot rule out session-recognition as a source of the scores; we do not claim that sessions are separable independently of attack traffic.

Consequence. The headline scores of §VI are feasibility evidence that PacketIn-level features distinguish InSDN's attack captures from its benign captures, not a measurement of attack-detection accuracy. A convincing evaluation needs benign and attack traffic interleaved within one session, with labels for all controller-visible traffic, which InSDN does not provide.

TABLE V
WITHHELD ATTACK FAMILY (BACC, W=1 S, STRICT LABELS). "BEST THR.": BEST OF FOUR THRESHOLDS, CHOSEN PER FAMILY.

Withheld family	MLP	Logistic	Best thr.
Flood (DoS+DDoS)	0.936±0.011	0.953	0.814
Probe	0.997±0.000	0.988	0.970
Web	0.904±0.020	0.909	0.699
Brute force	0.891±0.029	0.913	0.888
Exploit	0.929±0.037	0.997	0.929
Botnet	0.932±0.024	0.803	0.965

TABLE VI
CROSS-DEPLOYMENT TRANSFER (BACC, STRICT LABELS).

Train → test	MLP	Logistic	Best thr.
Metasploitable-2 → OVS	0.905±0.021	0.928	0.866
OVS → Metasploitable-2	0.990±0.001	0.985	0.947

VIII. A FORWARDING-APPLICATION CASE STUDY

InSDN shows one forwarding configuration. To see whether what the controller observes depends on the forwarding application, we built a small testbed (Mininet, one Open vSwitch switch, six hosts; a Ryu OpenFlow 1.3 learning switch with a configurable entry granularity) and recorded its control channel with the same parser. The three modes install, for each learned destination, an entry matching (i) the MAC pair; (ii) the 5-tuple for TCP/UDP with a 10 s idle timeout; or (iii) the 5-tuple with a 2 s idle timeout applied to all entries. In the two 5-tuple modes, other IP traffic receives an (src, dst, protocol) entry and ARP an ARP-only entry, so a coarse entry for one protocol cannot match another. The controller polls flow statistics every second. Traffic is benign only and generated by the same program with the same random seeds in every mode (request timing still differs): five hosts repeatedly fetch files of 16 KB to 1 MB from an HTTP server (80%) or ping it (20%) with random pacing for 90 s (captures span ∼99 s including start-up). Each mode was run 10 times (seeds 0–9), reporting mean and 95% t-interval over runs (Table VIII); the 30 runs completed 26 891 requests, of which 41 failed (below). We repeated the study on a loop-free three-switch chain (Table IX, below).

Granularity sets the PacketIn volume. MAC-pair mode produced 12.3 ± 2.3 TCP PacketIns per run, against $10\,422 \pm 202$ (10 s idle) and $10\,392 \pm 249$ (2 s idle) under per-flow forwarding: 665× to 1 752× (10 s idle) and 668× to 1 780× (2 s idle) more in each of the ten paired runs (larger in 10 of 10; two-sided sign-test $p = 0.002$). That is about 11–12 TCP PacketIns per completed request, so each connection is punted repeatedly; the most likely cause is packets that arrive before the FlowMod takes effect (we install entries without a barrier and send full packets), but we did not isolate this mechanism. FlowMods (19.1 vs. about 10 440 and 10 560 per run) track the PacketIns and are not independent evidence. The MAC-pair result is a replication: the same mode in the first, flawed campaign gave 11.0 ± 4.1. Seeds 0–4 are the earlier five runs per mode (reported in the previous version of this study); seeds 5–9 were added later.

TABLE VII
SESSION PROBE: PREDICTING BENIGN VS. ATTACK *session* FROM PACKETIN FEATURES AFTER REMOVING ATTACK-MATCHED PACKETINS (≥5 REMAINING PER WINDOW). THE RESIDUAL CAN STILL CONTAIN UNLISTED ATTACK TRAFFIC (D1: 424 OF 518 ATTACK-SESSION WINDOWS CONTAIN UNMATCHED-IP PACKETINS), AND D2 RESTS ON 6 WINDOWS FROM 2 CAPTURES.

Variant	BAcc	Majority
All windows (D1)	0.940±0.005	0.500
Attack-free windows only (D2)	0.950±0.031	0.500

Idle timeout sets what a poller sees. Per-flow entries returned per statistics reply fell from 153.1 ± 2.4 to 36.9 ± 0.5 when the idle timeout dropped from 10 s to 2 s, a ratio of 0.234 to 0.246 in all ten pairs: entries expire between polls, so a 1 s poller sees about a quarter as many. The result is close to what a shorter timeout implies; its value is the size of the loss for a realistic polling interval. In MAC-pair mode the replies contain only the table-miss rule and MAC-pair entries (9.5 ± 0.03 per reply); no entry matches IP or transport fields, as in InSDN (whose ONOS additionally installed `eth_type` punt rules).

A flawed first campaign. Our first application installed MAC-pair entries for ping and ARP in every mode; these also match later TCP between the same hosts, so most connections never reached the controller and per-flow mode produced only 43.4 ± 30.1 TCP PacketIns per run (4× MAC-pair) with a wide interval. The original application is archived as `ryu_fwd_v1.py`. We report it because it shows how an overlapping coarse entry hides traffic from the controller by two orders of magnitude, independently of the forwarding mode's nominal granularity.

Reliability. One run (5-tuple, 2 s idle, seed 4) had 41 failed requests (7–10 per client); the other 29 one-switch runs and all 30 three-switch runs had none. The run did not reproduce, and its failures were spread evenly over all five clients, which is consistent with a short stall of the shared path or controller rather than one client. The cause is not established (the failure reasons were not logged in that run; later runs log them). The run is included in all figures.

A second topology. On a three-switch chain (hosts h1–h6, server two hops from h1 and h2; 10 runs per mode, 26 826 requests, none failed) the contrast holds (Table IX): 18.0 ± 5.0 TCP PacketIns per run under MAC-pair against $10\,881 \pm 794$ and $10\,676 \pm 804$ under per-flow forwarding (391× to 1 509× in every paired run), and the per-flow entries a 1 s poller sees per reply fall from 112.4 ± 2.3 to 27.0 ± 0.4 (ratio 0.235 to 0.245). The ratio is smaller than on one switch because the MAC-pair baseline rises with the extra hops (18.0 vs. 12.3).

A stock Ryu application. To check that the MAC-pair behaviour is not an artefact of our own application, we ran Ryu's unmodified `simple_switch_13` on the same one-switch topology (5 runs of 60 s; 2 982 requests, none failed). Of 91 FlowMods, 86 match `(in_port, eth_dst, eth_src)` and 5 are the table-miss rule; none matches IP or transport fields. The controller received 71, 69, 76, 76 and 76 PacketIns per run (mean 73.6, all protocols). Like ONOS's `fwd`, this application installs MAC-pair entries; this follows from the

TABLE VIII
FORWARDING APPLICATION VS. WHAT THE CONTROLLER OBSERVES: BENIGN TRAFFIC FROM THE SAME GENERATOR AND SEEDS (5 HOSTS → 1 HTTP SERVER, PLUS PINGS), 1-SWITCH MININET/RYU TESTBED, ∼99 S CAPTURE PER RUN, 10 RUNS PER MODE, MEAN±95% CI. ENTRIES = FLOW ENTRIES RETURNED PER FLOW-STATISTICS REPLY (1 S POLLING).

Mode	TCP PacketIns	All PacketIns	FlowMods	MAC-pair entries	IP-specific entries
MAC-pair (as ONOS in InSDN)	12.3±2.3	76.8±1.8	19.1±1.8	9.5±0.0	0.0±0.0
5-tuple, 10 s idle	10422.0±202.0	10497.0±204.7	10442.8±200.8	0.0±0.0	153.1±2.4
5-tuple, 2 s idle	10391.6±249.4	10611.3±258.7	10556.4±252.3	0.0±0.0	36.9±0.5

TABLE IX
FORWARDING APPLICATION VS. WHAT THE CONTROLLER OBSERVES: BENIGN TRAFFIC FROM THE SAME GENERATOR AND SEEDS (5 HOSTS → 1 HTTP SERVER, PLUS PINGS), 1-SWITCH MININET/RYU TESTBED, ∼99 S CAPTURE PER RUN, 10 RUNS PER MODE, MEAN±95% CI. ENTRIES = FLOW ENTRIES RETURNED PER FLOW-STATISTICS REPLY (1 S POLLING).

Mode	TCP PacketIns	All PacketIns	FlowMods	MAC-pair entries	IP-specific entries
MAC-pair (as ONOS in InSDN)	18.0±5.0	267.8±18.2	33.9±3.8	7.0±0.0	0.0±0.0
5-tuple, 10 s idle	10881.3±793.8	11159.9±795.4	10928.3±791.8	0.0±0.0	112.4±2.3
5-tuple, 2 s idle	10675.5±803.9	11275.8±814.7	11040.4±805.9	0.0±0.0	27.0±0.4

application's code, and it does not test OpenDaylight, ONOS under other applications, or any production deployment.

Scope. This is a testbed with at most three switches with one forwarding application and benign traffic only; we run no attacks or detectors in it. The mode contrast does not transfer to InSDN's ONOS, which punted about one packet per target packet even with MAC-pair entries (§IV), and no claim is made about detection accuracy under any forwarding mode.

IX. DISCUSSION AND LIMITATIONS

Single testbed. All InSDN results come from one ONOS/OVS deployment with 44 controller captures; class counts are small for exploit, web, brute force and botnet. Confidence intervals reflect fold and initialisation variability, not new testbeds.

Forwarding-application dependence. The observability gap follows from ONOS reactive forwarding as configured in InSDN. An application installing 5-tuple entries would expose finer counters at higher flow-table cost, and the PacketIn stream would then be sparser. Our pipeline measures the gap for any capture; the numbers are specific to this configuration.

Labels. Strict labels leave brute force, exploit, web and DDoS with few attack windows because the CSV omits much of their traffic (for DDoS, most of the spoofed flood). An alternative rule that also counts IPs seen only in attack flows labels more windows but overlaps with the IP-count features, so we report it as a sensitivity analysis. Under that broader rule overall BAcc is lower (MLP 0.881, logistic regression 0.847, best threshold 0.748 at 1 s; MLP 0.887, best threshold 0.769 at 5 s) and within-session macro BAcc is 0.855 (0.782 at 5 s). The ranking against thresholds is unchanged, and at 1 s the MLP is ahead of the best threshold on flood, web and brute force, level on probe (0.812 vs. 0.794, within the interval) and behind on exploit and botnet. Transfer from Metasploitable-2 to OVS no longer beats the best threshold (0.678 vs. 0.678), while OVS to Metasploitable-2 remains strong (0.977 vs. 0.924).

Baselines and deployment. A GPU gradient-boosted tree baseline (XGBoost, BAcc 0.952 ± 0.013 at 5 s, strict labels, 10 seeds) does not beat the linear (0.961 ± 0.011) or MLP (0.958 ± 0.012) models; detection is run offline on recorded control channels, not inline in a live controller, and we do not measure controller-side latency or mitigation effects. The forwarding study (§VIII) uses benign traffic and at most three switches; it measures visibility, not detection, and a cross-testbed detection experiment with labelled attacks is future work.

Deployment. A controller-side detector of this kind would run beside the control channel, reading a port mirror or the controller's message log, and would need the forwarding application's granularity to be stable: switching from MAC-pair to per-flow forwarding changed TCP PacketIn volume by two to three orders of magnitude in our testbed, so any threshold or model tied to PacketIn rates must be retrained, or at least recalibrated, after a forwarding change. The idle timeout also changes what a poller sees (about a quarter as many per-flow entries at 2 s as at 10 s). Per-flow forwarding exposes finer counters at a controller-load and flow-table cost that we did not measure. Missing before deployment can be argued: interleaved benign and attack traffic in one session, a second controller family beyond Ryu applications (OpenDaylight, ONOS under other applications), and online latency.

What we could not do. We did not re-implement a published controller-side detector from its paper, so the comparison is to threshold heuristics and our own models only. The two public candidates we examined as second sources (Appendix A) could not be used.

X. CONCLUSION

InSDN's own controller never installed or reported the 5-tuple flow entries InSDN labels: its flow entries are MAC pairs or EtherType rules, and its view of traffic is a PacketIn stream. Restricted to that view, small models separate InSDN's attack captures from its benign captures on unseen captures, withheld families and a second deployment, where the usual PacketIn-rate heuristic does not (BAcc 0.946 vs.

0.616 on unseen captures). We could not show that this reflects attack detection rather than session recognition and label coverage: within sessions a trivial IP-PacketIn rule matches or beats the model under strict labels, and our probes lack clean attack-free windows. We therefore present controller-observable evaluation on InSDN as feasible but unproven, and recommend datasets that interleave benign and attack traffic. A benign-traffic Mininet/Ryu study shows that forwarding-application choices (entry granularity, idle timeout) change what the controller can observe. The pipeline, capture hashes and result files accompany this paper.

Appendix A
Candidate Second Datasets Examined

UKMDDoSDN [19]: we downloaded `syn_flood_combined.pcap` (18 922 240 B; 199 273 packets) and `udp_flood_combined.pcap` (15 420 748 B; 110 518 packets) from the authors' repository, parsed them (pcapng, Linux cooked v2) and found no packet on TCP port 6633 or 6653: they are data-plane captures from many per-host interfaces. The CSV files the dataset page describes were not in the repository listing and the Mendeley file list could not be read, so we make no claim about them. The dataset page states CC BY 4.0 while the repository is MIT-licensed; we did not resolve this. LAN-SDN-NIDS [10]: we read its Crossref record (the publisher page refused automated access), which describes a flow-level dataset of 1 125 059 records with 11 control-plane metrics; we could not verify whether raw OpenFlow captures are released, and flow-level counts of PacketIn and FlowMod would not expose FlowMod match fields. Neither was used as data. Hashes are in `data/MANIFEST.md`.

References

[1] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[2] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2017, pp. 253–262.

[3] M. S. Elsayed, N.-A. Le-Khac, and A. D. Jurcut, "InSDN: A novel SDN intrusion dataset," *IEEE Access*, vol. 8, pp. 165 263–165 284, 2020.

[4] Open Networking Foundation, "OpenFlow switch specification, version 1.3.0," ONF TS-006, 2012.

[5] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.

[6] D. Pinto, J. Vitorino, E. Maia, I. Amorim, and I. Praça, "Flow exporter impact on intelligent intrusion detection systems," in *Proc. 11th Int. Conf. Information Systems Security and Privacy (ICISSP)*, vol. 2, 2025, pp. 289–298.

[7] A. Pekar and R. Jozsa, "Evaluating ML-based anomaly detection across datasets of varied integrity: A case study," 2024, arXiv:2401.16843.

[8] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile Networks and Applications*, vol. 27, no. 1, pp. 357–370, 2022.

[9] A. Pekar and R. Jozsa, "Early-stage anomaly detection: A study of model performance on complete vs. partial flows," 2024, arXiv:2407.02856; accepted at WTMC 2025.

[10] J. Arévalo-Herrera, J. E. Camargo, J. I. Martínez Torre, J. M. Ramírez, and T. Zona-Ortiz, "A multi-class SDN intrusion detection dataset with synchronized OpenFlow control-plane telemetry," *Data*, vol. 11, no. 8, p. 195, 2026.

[11] R. Braga, E. Mota, and A. Passito, "Lightweight DDoS flooding attack detection using NOX/OpenFlow," in *Proc. IEEE Local Computer Network Conference (LCN)*, 2010, pp. 408–415.

[12] S. M. Mousavi and M. St-Hilaire, "Early detection of DDoS attacks against SDN controllers," in *Proc. Int. Conf. Computing, Networking and Communications (ICNC)*, 2015, pp. 77–81.

[13] S. Shin, V. Yegneswaran, P. Porras, and G. Gu, "AVANT-GUARD: Scalable and vigilant switch flow management in software-defined networks," in *Proc. ACM Conf. Computer and Communications Security (CCS)*, 2013, pp. 413–424.

[14] H. Wang, L. Xu, and G. Gu, "FloodGuard: A DoS attack prevention extension in software-defined networks," in *Proc. 45th Annual IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2015, pp. 239–250.

[15] M. Dhawan, R. Poddar, K. Mahajan, and V. Mann, "SPHINX: Detecting security attacks in software-defined networks," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2015.

[16] P. Berde, M. Gerola, J. Hart, Y. Higuchi, M. Kobayashi, T. Koide, B. Lantz, B. O'Connor, P. Radoslavov, W. Snow, and G. Parulkar, "ONOS: Towards an open, distributed SDN OS," in *Proc. 3rd Workshop on Hot Topics in Software Defined Networking (HotSDN)*, 2014, pp. 1–6.

[17] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.

[18] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium*, 2022, pp. 3971–3988.

[19] M. A. B. Mokti, "UKMDDoSDN: A DDoS attacks dataset for software-defined networks, version 2," Mendeley Data, 2025.